INTERPREP SERVER DOCUMENTATION

Real-Time Interview Preparation Platform - Server Section

Version: 1.0
Date: May 2026
Author: Swapnamoy Bhattacharya

TABLE OF CONTENTS

SECTION 1: CONFIGURATION FILES

1.1 Database Configuration (db.js)

File Path: `server/src/config/db.js`

Purpose

Establishes connection to MongoDB database using Mongoose ODM.

Code Block

```
const mongoose = require("mongoose");

// Connecting mongoDB database
const connectDB = async () => {
    try {
        await mongoose.connect(process.env.MONGO_URI);
        console.log("MongoDB connected");
    } catch (error) {
        console.error("DB connection failed:", error.message);
        process.exit(1);
    }
};

module.exports = connectDB;
```

Breakdown

- **Mongoose Connection:** Connects to MongoDB using the connection string from environment variables
- **Error Handling:** If connection fails, the server process exits with code 1
- **Input Required:** `MONGO_URI` from `.env` file

Output

- Returns connection function that connects to MongoDB
- Logs success/failure messages to console

1.2 Environment Configuration (.env)

File Path: `server/.env`

Purpose

Stores sensitive configuration and environment-specific variables.

Variables Defined

Variable	Description
MONGO_URI	MongoDB connection string
ACCESS_TOKEN_SECRET	Secret key for JWT access tokens
REFRESH_TOKEN_SECRET	Secret key for JWT refresh tokens
EMAIL_USER	Gmail account for sending emails
EMAIL_PASS	App password for Gmail SMTP
CLIENT_URL	Frontend application URL

SECTION 2: MODELS (MONGODB SCHEMAS)

2.1 User Model (user.model.js)

File Path: `server/src/models/user.model.js`

Purpose

Defines the user schema for authentication and authorization.

Code Block

```
const mongoose = require("mongoose");
const bcrypt = require("bcrypt");

// Defining the user schema
const userSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true,
        unique: true
    },
    password: {
        type: String,
        required: true
    },
    role: {
        type: String,
        enum: ["user", "admin"],
        default: "user"
    },
    refreshToken: {
        type: String
    },
    isVerified: {
        type: Boolean,
        default: false
    },
    verificationToken: String,
    verificationTokenExpiry: Date,
    resetPasswordToken: String,
    resetPasswordExpiry: Date
});

// pre("save") middleware
userSchema.pre("save", async function () {
    // Only hash if password is modified
    if(!this.isModified("password")) return;

    // Salting and Hashing the password
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
});

// Checks whether a password entered by an user during login matches the password stored in the database
userSchema.methods.comparePassword = async function (enteredPassword) {
    return await bcrypt.compare(enteredPassword, this.password);
}

module.exports = mongoose.model("User", userSchema);
```

Breakdown

Field	Type	Purpose
name	String	User's display name
email	String	Unique email address
password	String	Hashed password
role	String	"user" or "admin"
refreshToken	String	Stored refresh token for session
isVerified	Boolean	Email verification status
verificationToken	String	Token for email verification
resetPasswordToken	String	Token for password reset

Pre-save Middleware

- Automatically hashes password before saving (using bcrypt with salt factor 10)
- Only hashes if password is modified

Methods

- **comparePassword():** Compares entered password with stored hashed password

---

2.2 Problem Model (problem.model.js)

File Path: `server/src/models/problem.model.js`

Purpose

Defines the schema for coding problems in the platform.

Code Block

```
const mongoose = require("mongoose");

const problemSchema = new mongoose.Schema({
    title: {
        type: String,
        required: true,
        trim: true
    },
    description: {
        type: String,
        required: true
    },
    category: {
        type: String,
        enum: ["DSA", "HR", "System Design"],
        required: true
    },
    difficulty: {
        type: String,
        enum: ["Easy", "Medium", "Hard"],
        required: true
    },
    tags: [{
        type: String
    }],
    constraints: String,
    examples: [
        {
            input: String,
            output: String,
            explanation: String
        }
    ],
    createdBy: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "User",
        required: true
    },
    testCases: [
        {
            input: String,
            output: String
        }
    ]
}, {timestamps: true});

module.exports = mongoose.model("Problem", problemSchema);
```

Breakdown

Field	Type	Purpose
title	String	Problem title
description	String	Problem description
category	String	Category (DSA, HR, System Design)
difficulty	String	Easy/Medium/Hard
tags	Array	Problem tags
constraints	String	Problem constraints
examples	Array	Example inputs/outputs
createdBy	ObjectId	Admin who created problem
testCases	Array	Test cases for code execution

2.3 Submission Model (submission.model.js)

File Path: `server/src/models/submission.model.js`

Purpose

Tracks user submissions and their evaluation results.

Code Block

```
const mongoose = require("mongoose");

const submissionSchema = new mongoose.Schema({
    user: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "User",
        required: true
    },
    problem: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Problem",
        required: true
    },
    code: {
        type: String,
        required: true
    },
    language: {
        type: String,
        enum: ["java", "javascript", "python", "cpp"],
        required: true
    },
    status: {
        type: String,
        enum: [
            "Pending",
            "Accepted",
            "Wrong Answer",
            "Runtime Error",
            "Time Limit Exceeded"
        ],
        default: "Pending"
    },
    output: String,
    error: String,
    executionTime: Number
}, { timestamps: true });

module.exports = mongoose.model("Submission", submissionSchema);
```

Breakdown

Field	Type	Purpose
user	ObjectId	Reference to User model
problem	ObjectId	Reference to Problem model
code	String	Submitted code
language	String	Programming language used
status	String	Submission status
output	String	Code execution output
error	String	Error message if any
executionTime	Number	Execution time in milliseconds

SECTION 3: CONTROLLERS

3.1 Authentication Controller (auth.controller.js)

File Path: `server/src/controllers/auth.controller.js`

Purpose

Handles all authentication-related operations including registration, login, email verification, and password management.

Functions Overview

3.1.1 Register Function

```
const register = asyncHandler ( async (req, res) => {
    const { name, email, password, role } = req.body;

    // Check for existing user
    const existingUser = await User.findOne({ email });

    if (existingUser) {
        throw new ApiError(status.BAD_REQUEST, "User already exists");
    }

    const verificationToken = generateVerificationToken();

    // Save the user
    const user = new User({
        name,
        email,
        password,
        role,
        verificationToken,
        verificationTokenExpiry: Date.now() + 10 * 60 * 1000
    });

    await user.save();

    await sendVerificationEmail(email, verificationToken);

    return res.status(status.CREATED).json({
        message: "User registered successfully. Please verify your email.",
        user: {
            id: user._id,
            name: user.name,
            email: user.email,
            role: user.role
        }
    });
});
```

Purpose: Registers a new user and sends verification email
Input: name, email, password, role (optional)
Output: User object with ID, name, email, role

3.1.2 Resend Verification Function

```
const resendVerification = asyncHandler(async (req, res) => {
    const { email } = req.body;

    const user = await User.findOne({ email });

    if (!user) {
        throw new ApiError(status.NOT_FOUND, "User not found");
    }

    if (user.isVerified) {
        throw new ApiError(status.BAD_REQUEST, "User already verified");
    }

    const token = generateVerificationToken();

    user.verificationToken = token;
    user.verificationTokenExpiry = Date.now() + 10 * 60 * 1000;

    await user.save();

    await sendVerificationEmail(email, token);

    res.json({ message: "Verification email resent" });
});
```

Purpose: Resends verification email to unverified users
Input: email
Output: Success message

3.1.3 Verify Email Function

```
const verifyEmail = asyncHandler( async (req, res) => {
    const { token } = req.query;

    if(!token) {
        throw new ApiError(status.BAD_REQUEST, "Verification token is required");
    }

    const user = await User.findOne({
        verificationToken: token,
        verificationTokenExpiry: { $gt: Date.now() }
    });

    if(!user) {
        throw new ApiError(status.UNAUTHORIZED, "Expired or invalid verification token");
    }

    user.isVerified = true;
    user.verificationToken = null;
    user.verificationTokenExpiry = null;

    await user.save();

    // Generate tokens for auto-login
    const accessToken = tokenService.generateAccessToken(user);
    const refreshToken = tokenService.generateRefreshToken(user._id);

    user.refreshToken = refreshToken;
    await user.save();

    return res.status(status.SUCCESS).json({
        message: "Email verified successfully. Logged in automatically.",
        accessToken,
        refreshToken,
        user: { id: user._id, name: user.name, email: user.email, role: user.role }
    });
});
```

Purpose: Verifies user email and provides auto-login tokens
Input: verification token from query parameter
Output: accessToken, refreshToken, user object

3.1.4 Forgot Password Function

```
const forgotPassword = asyncHandler ( async (req, res) => {
    const { email } = req.body;

    const user = await User.findOne({ email });
    if(!user) {
        throw new ApiError(status.UNAUTHORIZED, "Invalid credential");
    }

    if(user.refreshToken) {
        throw new ApiError(status.BAD_REQUEST, "User already logged in");
    }

    const token = generateResetPasswordToken();

    user.resetPasswordToken = token;
    user.resetPasswordExpiry = Date.now() + 10 * 60 * 1000;

    await user.save();

    sendResetPasswordEmail(email, token);
    res.json({ message: "Reset link sent to email" });
});
```

Purpose: Initiates password reset process
Input: email
Output: Success message (does not reveal if user exists)

3.1.5 Reset Password Function

```
const resetPassword = asyncHandler ( async (req, res) => {
    const { token, newPassword } = req.body;

    if(!token || !newPassword) {
        throw new ApiError(status.BAD_REQUEST, "Token and new password required");
    }

    const user = await User.findOne({
        resetPasswordToken: token,
        resetPasswordExpiry: { $gt: Date.now() }
    });

    if(!user) {
        throw new ApiError(status.BAD_REQUEST, "Invalid or expired token");
    }

    user.resetPasswordToken = null;
    user.resetPasswordExpiry = null;
    user.password = newPassword;

    await user.save();

    res.json({ message: "Password reset successful" });
});
```

Purpose: Resets user password using valid token
Input: token, newPassword
Output: Success message

3.1.6 Login Function

```
const logIn = asyncHandler ( async (req, res) => {
    const { email, password } = req.body;

    const user = await User.findOne({ email });

    if (!user) {
        throw new ApiError(status.UNAUTHORIZED, "Invalid credentials");
    }

    if(!user.isVerified) {
        throw new ApiError(status.FORBIDDEN, "Please verify your email first");
    }

    const isMatch = await user.comparePassword(password);
    if (!isMatch) {
        throw new ApiError(status.UNAUTHORIZED, "Invalid credentials");
    }

    const accessToken = tokenService.generateAccessToken(user);
    const refreshToken = tokenService.generateRefreshToken(user._id);

    user.refreshToken = refreshToken;
    await user.save();

    return res.status(status.SUCCESS).json({
        message: "Log in successfull",
        accessToken,
        refreshToken,
        user: { id: user._id, name: user.name, email: user.email, role: user.role }
    });
});
```

Purpose: Authenticates user and provides access/refresh tokens
Input: email, password
Output: accessToken, refreshToken, user object

3.1.7 Refresh Token Handler

```
const refreshTokenHandler = asyncHandler ( async (req, res) => {
    const { refreshToken } = req.body;
    if (!refreshToken) {
        throw new ApiError(status.UNAUTHORIZED, "Refresh token not found");
    }

    let decoded;
    try {
        decoded = jwt.verify(refreshToken, process.env.REFRESH_TOKEN_SECRET);
    } catch (err) {
        throw new ApiError(status.UNAUTHORIZED, "Invalid or expired refresh token");
    }

    const user = await User.findById(decoded.id);

    if (!user || user.refreshToken !== refreshToken) {
        throw new ApiError(status.UNAUTHORIZED, "Invalid refresh token");
    }

    const accessToken = tokenService.generateAccessToken(user);

    return res.json({ newAccessToken: accessToken });
});
```

Purpose: Generates new access token using valid refresh token
Input: refreshToken
Output: newAccessToken

3.1.8 Logout Function

```
const logout = asyncHandler( async (req, res) => {
    const { refreshToken } = req.body;
    if (!refreshToken) {
        throw new ApiError(status.UNAUTHORIZED, "Refresh token not found");
    }

    const user = await User.findOne({ refreshToken });

    if (!user || user.refreshToken !== refreshToken) {
        throw new ApiError(status.UNAUTHORIZED, "Invalid refresh token");
    }

    user.refreshToken = null;
    await user.save();

    res.json({ message: "Logged out successfully" });
});
```

Purpose: Logs out user by clearing refresh token
Input: refreshToken
Output: Success message

3.2 Problem Controller (problem.controller.js)

File Path: `server/src/controllers/problem.controller.js`

Purpose

Handles problem creation and retrieval operations.

3.2.1 Create Problem Function

```
const createProblem = asyncHandler ( async (req, res) => {
    const problem = new Problem({
        ...req.body,
        createdBy: req.user.id
    });

    await problem.save();

    res.status(status.CREATED).json({
        success: true,
        data: problem
    });
});
```

Purpose: Creates a new problem (admin only)
Input: Problem data from request body, user ID from auth middleware
Output: Created problem object

3.2.2 Get Problems Function

```
const getProblems = asyncHandler ( async (req, res) => {
    const { difficulty, category, page = 1, limit = 10 } = req.query;

    const query = {};
    if(difficulty) query.difficulty = difficulty;
    if(category) query.category = category;

    const problems = await Problem.find(query)
        .skip((page - 1) * limit)
        .limit(Number(limit));

    res.status(status.SUCCESS).json({
        success: true,
        count: problems.length,
        data: problems
    });
});
```

Purpose: Retrieves problems with optional filtering and pagination
Input: Query parameters (difficulty, category, page, limit)
Output: Array of problems with count

3.3 Submission Controller (submission.controller.js)

File Path: `server/src/controllers/submission.controller.js`

Purpose

Handles code submission and execution evaluation.

Create Submission Function

```
const createSubmission = asyncHandler(async (req, res) => {
    const { problem: problemId, code, language } = req.body;

    if (!problemId) {
        throw new ApiError(status.BAD_REQUEST, "problem is required");
    }

    const problem = await Problem.findById(problemId);

    if (!problem) {
        throw new ApiError(404, "Problem not found");
    }

    // Create submission (pending)
    let submission = await Submission.create({
        user: req.user.id,
        problem: problemId,
        code,
        language,
        status: "Pending"
    });

    // Run evaluation
    const result = await runner.runCode({
        code,
        testCases: problem.testCases,
        language
    });

    // Save the result
    submission.status = result.status;
    submission.output = result.output;
    submission.error = result.error || null;

    await submission.save();

    res.status(status.CREATED).json({
        success: true,
        data: submission
    });
});
```

Purpose: Creates submission, runs code against test cases, and returns result
Input: problemId, code, language
Output: Submission object with status, output, error

SECTION 4: MIDDLEWARE

4.1 Auth Middleware (auth.middleware.js)

File Path: `server/src/middleware/auth.middleware.js`

Purpose

Protects routes by verifying JWT access tokens.

Code Block

```
const jwt = require("jsonwebtoken");
const status = require("../constants/statusCodes");
const ApiError = require("../utils/apiError");

// Authentication middleware for protected routs
const authMiddleware = (req, res, next) => {
    const authHeader = req.headers.authorization;

    // check if header exists
    if (!authHeader) {
        return next(new ApiError(status.UNAUTHORIZED, "No token provided"));
    }

    // Check token format
    if (!authHeader.startsWith("Bearer ")) {
        return next(new ApiError(status.UNAUTHORIZED, "Invalid token format"));
    }

    try {
        // Accessing the token
        const token = authHeader.split(" ")[1];

        // Verify token
        const decoded = jwt.verify(token, process.env.ACCESS_TOKEN_SECRET);

        // Attach user info to request
        req.user = decoded;

        next();
    } catch (error) {
        return next(new ApiError(status.UNAUTHORIZED, "Invalid or expired token"));
    }
}

module.exports = authMiddleware;
```

Breakdown

- **Input:** Authorization header with Bearer token
- **Output:** Adds decoded user info to `req.user` object
- **Validation:** Checks token format and verifies against ACCESS_TOKEN_SECRET

4.2 Role Middleware (role.middleware.js)

File Path: `server/src/middleware/role.middleware.js`

Purpose

Authorizes users based on their roles (admin/user).

Code Block

```
const status = require("../constants/statusCodes");
const ApiError = require("../utils/apiError");

const authorizeRoles = (...allowedRoles) => {
    return (req, res, next) => {
        // 1. Check roles config
        if (allowedRoles.length === 0) {
            return next(new ApiError(status.INTERNAL_SERVER_ERROR, "No roles specified for authorization"));
        }

        // Check if the user exists (set by auth middleware)
        if(!req.user) {
            return next(new ApiError(status.UNAUTHORIZED, "Unauthorized access"));
        }

        // Check role exists
        if (!req.user.role) {
            return next(new ApiError(status.UNAUTHORIZED, "User role not found"));
        }

        // Check if the user's role is allowed
        if(!allowedRoles.includes(req.user.role)) {
            return next(new ApiError(status.FORBIDDEN, `Role '${req.user.role}' is not allowed`));
        }

        next();
    }
}

module.exports = authorizeRoles;
```

Breakdown

- **Input:** Multiple role strings (e.g., "admin", "user")
- **Output:** Continues to next middleware if role is authorized
- **Validation:** Checks if user's role is in allowed roles list

---

4.3 Error Middleware (error.middleware.js)

File Path: `server/src/middleware/error.middleware.js`

Purpose

Global error handler that catches and formats all errors.

Code Block

```
const logger = require("../utils/logger");

// Global error handler middleware
const errorHandler = (err, req, res, next) => {
    const statusCode = err.statusCode || 500;

    // Log error using Winston
    logger.error({
        message: err.message,
        statusCode,
        method: req.method,
        url: req.originalUrl,
        stack: err.stack,
        user: req.user ? req.user.id : null
    });

    res.status(statusCode).json({
        success: false,
        message: err.message || "Internal server error",
        stack: process.env.NODE_ENV === "development" ? err.stack : undefined
    });
}

module.exports = errorHandler;
```

Breakdown

- **Input:** Error object from try-catch or next(error)
- **Output:** JSON response with error message and status code
- **Logging:** Uses Winston logger for error tracking
- **Stack Trace:** Only shown in development mode

4.4 Validation Middleware (validate.middleware.js)

File Path: `server/src/middleware/validate.middleware.js`

Purpose

Validates request body against Joi schemas.

Code Block

```
const ApiError = require("../utils/apiError");
const status = require("../constants/statusCodes");

// Reusable Validation Middleware
const validate = (schema) => (req, res, next) => {
    const { error } = schema.validate(req.body, {
        abortEarly: false // Show all errors
    });

    if(error) {
        const message = error.details.map(err => err.message).join(", ");
        return next(new ApiError(status.BAD_REQUEST, message));
    }

    next();
};

module.exports = validate;
```

Breakdown

- **Input:** Joi validation schema
- **Output:** Passes to next middleware if valid, otherwise returns validation errors
- **Validation:** Returns all errors (abortEarly: false)

SECTION 5: SERVICES

5.1 Token Service (token.service.js)

File Path: `server/src/services/token.service.js`

Purpose

Generates JWT access and refresh tokens.

Code Block

```
const jwt = require("jsonwebtoken");

// Generating the access token using JWT
const generateAccessToken = (user) => {
    return jwt.sign(
        { id: user._id, role: user.role },
        process.env.ACCESS_TOKEN_SECRET,
        { expiresIn: "15m" }
    );
};

const generateRefreshToken = (userId) => {
    return jwt.sign(
        { id: userId },
        process.env.REFRESH_TOKEN_SECRET,
        { expiresIn: "7d" }
    )
}

module.exports = { generateAccessToken, generateRefreshToken };
```

Breakdown

Function	Input	Output	Expiry
generateAccessToken	user object	JWT token	15 minutes
generateRefreshToken	userId	JWT token	7 days

5.2 Runner Service (runner.service.js)

File Path: `server/src/services/runner.service.js`

Purpose

Executes user-submitted code against test cases and evaluates correctness.

Code Block

```
const { spawn } = require("child_process");
const fs = require("fs");
const path = require("path");

const getExecutionDetails = (language, filePath) => {
```

```
    switch (language) {
        case "javascript":
            return { command: "node", args: [filePath] };
        case "python":
            return { command: "python", args: [filePath] };
        case "java":
            return {
                compile: { command: "javac", args: [filePath] },
                run: { command: "java", args: ["-cp", path.dirname(filePath), "Main"] }
            };
        default:
            throw new Error("Unsupported language");
    }
};

const runCode = async ({ code, testCases, language }) => {
    const fileName = `temp-${Date.now()}`;
    let filePath;

    // Assign extension
    if (language === "javascript") filePath = path.join(__dirname, `${fileName}.js`);
    if (language === "python") filePath = path.join(__dirname, `${fileName}.py`);
    if (language === "java") filePath = path.join(__dirname, `Main.java`);

    fs.writeFileSync(filePath, code);

    const execDetails = getExecutionDetails(language, filePath);

    // Compile (Java only)
    if (language === "java") {
        const compileResult = await new Promise((resolve) => {
            const compile = spawn(execDetails.compile.command, execDetails.compile.args);
            let error = "";
            compile.stderr.on("data", (data) => { error += data.toString(); });
            compile.on("close", () => {
                if (error) return resolve({ status: "Compilation Error", error });
                resolve({ success: true });
            });
        });
        if (compileResult.status === "Compilation Error") {
            fs.unlinkSync(filePath);
            return compileResult;
        }
    }

    // Run for each test case
    for (let test of testCases) {
        const result = await new Promise((resolve) => {
            const process = spawn(
                execDetails.run ? execDetails.run.command : execDetails.command,
                execDetails.run ? execDetails.run.args : execDetails.args
            );

            let output = "";
            let error = "";

            process.stdout.on("data", (data) => { output += data.toString(); });
            process.stderr.on("data", (data) => { error += data.toString(); });

            process.stdin.write(test.input + "\n");
            process.stdin.end();

            const timeout = setTimeout(() => {
                process.kill();
                resolve({ status: "Time Limit Exceeded", error: "Execution timed out" });
            }, 2000);

            process.on("close", () => {
                clearTimeout(timeout);
                if (error) return resolve({ status: "Runtime Error", error });
                resolve({ output: output.trim() });
            });
        });

        if (result.status === "Runtime Error" || result.status === "Time Limit Exceeded") {
            cleanup(filePath, language);
            return result;
        }

        if (result.output !== test.output.trim()) {
            cleanup(filePath, language);
            return { status: "Wrong Answer", output: result.output };
        }
    }

    cleanup(filePath, language);
```

```
    return { status: "Accepted" };
};

const cleanup = (filePath, language) => {
    if (fs.existsSync(filePath)) fs.unlinkSync(filePath);
    if (language === "java") {
        const classFile = path.join(path.dirname(filePath), "Main.class");
        if (fs.existsSync(classFile)) fs.unlinkSync(classFile);
    }
};

module.exports = { runCode };
```

Breakdown

Aspect	Details
Supported Languages	JavaScript, Python, Java
Execution Method	Child process spawning
Test Case Execution	Sequential execution against all test cases
Timeout	2 seconds per test case
Java Compilation	Compiles before execution
Cleanup	Removes temp files after execution

Input

```
{
    code: "string",
    testCases: [{ input: "string", output: "string" }],
    language: "javascript" | "python" | "java"
}
```

Output

```
{
    status: "Accepted" | "Wrong Answer" | "Runtime Error" | "Compilation Error" | "Time Limit Exceeded",
    output: "string",
    error: "string"
}
```

5.3 Email Service (email.service.js)

File Path: `server/src/services/email.service.js`

Purpose

Sends transactional emails (verification, password reset) using nodemailer.

Code Block

```
const nodemailer = require("nodemailer");

// Create transporter once and reuse it
const getTransporter = () => {
    return nodemailer.createTransport({
        service: "gmail",
        auth: {
            user: process.env.EMAIL_USER?.trim(),
            pass: process.env.EMAIL_PASS?.trim()
        }
    });
};

// Generic email sending function
const sendEmail = async (email, subject, html) => {
    const transporter = getTransporter();

    try {
        const info = await transporter.sendMail({
            from: process.env.EMAIL_USER?.trim(),
            to: email,
            subject,
            html
        });
        console.log("Email sent:", info.response);
        return info;
    } catch (error) {
        console.error("Email send error:", error.message);
        throw error;
    }
};

// Send verification email
const sendVerificationEmail = async (email, token) => {
    const verificationUrl = `${process.env.CLIENT_URL}/api/auth/verify-email?token=${token}`;
    const html = `
        <h2>Email Verification</h2>
        <p>Click below to verify your account:</p>
        <a href="${verificationUrl}">Verify Email</a>
    `;
    return sendEmail(email, "Verify your email", html);
};

// Send reset password email
const sendResetPasswordEmail = async (email, token) => {
    const resetUrl = `${process.env.CLIENT_URL}/api/auth/reset-password?token=${token}`;
    const html = `
        <h2>Password Reset</h2>
        <p>Click below to reset your password:</p>
        <a href="${resetUrl}">Reset Password</a>
        <p>This link expires in 10 minutes.</p>
    `;
    return sendEmail(email, "Reset your password", html);
};

module.exports = { sendVerificationEmail, sendResetPasswordEmail, sendEmail };
```

Breakdown

Function	Purpose
getTransporter	Creates nodemailer transporter with Gmail SMTP
sendEmail	Generic email sending function
sendVerificationEmail	Sends email verification link
sendResetPasswordEmail	Sends password reset link

SECTION 6: ROUTES

6.1 Auth Routes (auth.routes.js)

File Path: `server/src/routes/auth.routes.js`

Purpose

Defines all authentication-related API endpoints.

Code Block

```
const express = require("express");
const router = express.Router();
const authController = require("../controllers/auth.controller");
const validate = require("../middleware/validate.middleware");
const {
    registerSchema,
    resendVerificationSchema,
    loginSchema,
    refreshTokenSchema,
    logoutSchema
} = require("../utils/validators/auth.validator");

// API routs
router.post("/register", validate(registerSchema), authController.register);
router.post("/resend-verification", validate(resendVerificationSchema), authController.resendVerification);
router.post("/login", validate(loginSchema), authController.logIn);

router.post("/refresh", validate(refreshTokenSchema), authController.refreshTokenHandler);
router.post("/logout", validate(logoutSchema), authController.logout);

router.get("/verify-email", authController.verifyEmail);
router.post("/forgot-password", authController.forgotPassword);
router.post("/reset-password", authController.resetPassword);

module.exports = router;
```

Endpoints Summary

Method	Endpoint	Controller Function	Validation
POST	/register	register	registerSchema
POST	/resend-verification	resendVerification	resendVerificationSchema
POST	/login	logIn	loginSchema
POST	/refresh	refreshTokenHandler	refreshTokenSchema
POST	/logout	logout	logoutSchema
GET	/verify-email	verifyEmail	None
POST	/forgot-password	forgotPassword	None
POST	/reset-password	resetPassword	None

6.2 Problem Routes (problem.routes.js)

File Path: `server/src/routes/problem.routes.js`

Purpose

Defines API endpoints for problem management.

Code Block

```
const express = require("express");
const router = express.Router();

const controller = require("../controllers/problem.controller");
const authRoles = require("../middleware/role.middleware");
const authHeader = require("../middleware/auth.middleware");

router.post("/", authHeader, authRoles("admin"), controller.createProblem);
router.get("/", authHeader, controller.getProblems);

module.exports = router;
```

Endpoints Summary

Method	Endpoint	Access	Description
POST	/	Admin Only	Create new problem
GET		Authenticated	Get all problems (paginated)

6.3 Submission Routes (submission.routes.js)

File Path: `server/src/routes/submission.routes.js`

Purpose

Defines API endpoints for code submission.

Code Block

```
const express = require("express");
const router = express.Router();

const controller = require("../controllers/submission.controller");
const authHeader = require("../middleware/auth.middleware");

router.post("/", authHeader, controller.createSubmission);

module.exports = router;
```

Endpoints Summary

Method	Endpoint	Access	Description
POST	/	Authenticated	Submit code for evaluation

SECTION 7: UTILITIES

7.1 API Response & Error Utilities

7.1.1 API Error (apiError.js)

```
// Custom Error class
class ApiError extends Error {
    constructor(statusCode, message) {
        super(message);
        this.statusCode = statusCode;
    }
}

module.exports = ApiError;
```

Purpose: Custom error class with HTTP status code

7.2 Async Handler (asyncHandler.js)

```
// Forwards to the global middleware automatically
const asyncHandler = (fn) => {
    return (req, res, next) => {
        Promise.resolve(fn(req, res, next)).catch(next);
    }
}

module.exports = asyncHandler;
```

Purpose: Wraps async route handlers to catch errors and pass to error middleware

---

7.3 Validators (auth.validator.js)

File Path: `server/src/utils/validators/auth.validator.js`

Purpose

Defines Joi validation schemas for authentication endpoints.

Code Block

```
const joi = require("joi");

// Register validation
const registerSchema = joi.object({
    name: joi.string().min(3).max(30).required(),
    email: joi.string().email().required(),
    password: joi.string().min(6).max(20).required(),
    role: joi.string().valid("admin", "user").optional().default("user")
});

const resendVerificationSchema = joi.object({
    email: joi.string().email().required()
});

const loginSchema = joi.object({
    email: joi.string().email().required(),
    password: joi.string().required(),
});

const refreshTokenSchema = joi.object({
    refreshToken: joi.string().required()
});

const logoutSchema = joi.object({
    refreshToken: joi.string().required()
});

module.exports = {
    registerSchema,
    resendVerificationSchema,
    loginSchema,
    refreshTokenSchema,
    logoutSchema
};
```

Schema Breakdown

Schema	Validations
registerSchema	name: 3-30 chars, email: valid format, password: 6-20 chars, role: admin/user
resendVerificationSchema	email: valid format, required
loginSchema	email: valid format, password: required
refreshTokenSchema	refreshToken: required
logoutSchema	refreshToken: required

7.4 Token Utilities (token.utils.js)

File Path: `server/src/utils/token.utils.js`

Purpose

Generates cryptographic tokens for email verification and password reset.

Code Block

```
const crypto = require("crypto");

// Generate email verification token
const generateVerificationToken = () => {
    return crypto.randomBytes(32).toString("hex");
};

// Generate reset password token
const generateResetPasswordToken = () => {
    return crypto.randomBytes(32).toString("hex");
}

module.exports = { generateVerificationToken, generateResetPasswordToken };
```

Purpose: Generates 32-byte random hex tokens for secure token generation

---

7.5 Logger (logger.js)

Purpose: Winston-based logging for error tracking (referenced in error middleware)

SECTION 8: CONSTANTS

8.1 Status Codes (statusCodes.js)

File Path: `server/src/constants/statusCodes.js`

Purpose

Defines HTTP status codes used throughout the application.

Code Block

```
const STATUS = {
    SUCCESS: 200,
    CREATED: 201,
    BAD_REQUEST: 400,
    UNAUTHORIZED: 401,
    FORBIDDEN: 403,
    NOT_FOUND: 404,
    INTERNAL_SERVER_ERROR: 500
}

module.exports = STATUS;
```

8.2 Roles (roles.js)

File Path: `server/src/constants/roles.js`

Purpose

Defines user roles in the system.

Code Block

```
const ROLES = {
    USER: "user",
    ADMIN: "admin"
};

module.exports = ROLES;
```

SECTION 9: MAIN APPLICATION FILES

9.1 App Entry Point (app.js)

File Path: `server/app.js`

Purpose

Main Express application configuration and middleware setup.

Code Block

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const morgan = require("morgan");
const connectDB = require("./src/config/db");
const authRoutes = require("./src/routes/auth.routes");
const problemRouts = require("./src/routes/problem.routes");
const submissionRoute = require("./src/routes/submission.routes");
const errorHandler = require("./src/middleware/error.middleware");

const app = express();

// Initialization of database
connectDB();

app.use(cors({
    origin: "http://localhost:5173"
}));

// Middleware
app.use(express.json());
app.use(morgan("dev"));

// auth route
app.use("/api/auth", authRoutes);

// Problem route
app.use("/api/problem", problemRouts);

// Submission route
app.use("/api/submissions", submissionRoute);

// Calling error handling middleware
app.use(errorHandler);

module.exports = app;
```

Breakdown

Component	Description
Database	Connects to MongoDB on startup
CORS	Allows requests from http://localhost:5173
Body Parser	Parses JSON request bodies
Morgan	HTTP request logging
Routes	Mounts /api/auth, /api/problem, /api/submissions
Error Handler	Global error handling middleware

9.2 Server Entry Point (server.js)

File Path: `server/server.js`

Purpose

Starts the Express server on specified port.

Code Block

```
const app = require("./app");

// Initialization of server
const port = 3000;

app.listen(port, () => {
    console.log(`Server is running in ${port} port`);
});
```

Breakdown

- **Port:** 3000
- **Import:** Express app from app.js
- **Output:** Logs server startup message

APPENDIX: DEPENDENCIES

Package.json Dependencies

Package	Version	Purpose
bcrypt	^6.0.0	Password hashing
cors	^2.8.6	Cross-origin resource sharing
dotenv	^17.3.1	Environment variable management
express	^5.2.1	Web framework
joi	^18.1.2	Input validation
jsonwebtoken	^9.0.3	JWT token handling
mongoose	^9.2.4	MongoDB ODM
morgan	^1.10.1	HTTP request logging
nodemailer	^8.0.5	Email sending
winston	^3.19.0	Logging library

API ENDPOINT SUMMARY

Base URL

`http://localhost:3000/api`

Authentication Endpoints

Method	Endpoint	Description
POST	/auth/register	Register new user
POST	/auth/login	User login
POST	/auth/logout	User logout
GET	/auth/verify-email	Verify email address
POST	/auth/forgot-password	Request password reset
POST	/auth/reset-password	Reset password
POST	/auth/refresh	Refresh access token
POST	/auth/resend-verification	Resend verification email

Problem Endpoints

Method	Endpoint	Description
GET	/problem	Get all problems (with pagination)
POST	/problem	Create new problem (admin only)

Submission Endpoints

Method	Endpoint	Description

POST /submissions Submit code for evaluation

END OF DOCUMENTATION

For questions or updates, contact the development team.